

ALLIANCE UNIVERSITY

School of Advanced Computing

Department of Data Science

Mini Project Report

Project Title:

DNA SPLICE SITE CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORKS

Group Members:

1. VIJETA ALAVANI (11)

2. DIYA MATHEW (23)

Date of Submission:

20-08-2026

ALLIANCE UNIVERSITY

CERTIFICATE

This is to certify that the Mini Project work entitled **"DNA SPLICE SITE CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORKS"** is the Bonafide work done by VIJETA ALAVANI (11) and DIYA MATHEW (23) submitted for the internal mark requirements for the course Z1AMC 701 Data Science, Masters of Science in Data Science during the year of August 2026.

Dr. Jayasheelan Palaisamy

Supervisor & Assistant Professor, Data Science

TABLE OF CONTENTS

S.NO	Chapters	Page No.
1	Introduction	1-2
2	Literature Survey	2-8
3	Methodology	8-9
4	Implementation – Code Snippets	9-16
5	Results and Discussion	16-28
6	Conclusion	28-30
7	References	30-33

1. Introduction:

DNA splice site classification is a fundamental problem in bioinformatics and computational biology. Splice sites are short DNA sequences that indicate the start and end sites of intron removal from precursor mRNA (pre-mRNA) in the processes of RNA splicing. The correct identification of splice sites is essential for gene prediction and annotation. In particular, one of the main challenges in computational biology is the problem of classifying DNA sequences as acceptor or donor sites. Previously, various classification techniques based on machine learning algorithms have been used for this task, but achieving high accuracy is a difficult task due to the complexity of DNA sequence structure. However, with the appearance of deep learning technologies, such as Convolutional Neural Networks, it became possible to automatically extract the features inherent in each type of DNA sequence, thereby significantly increasing the accuracy of classifying these sequences.

Deep learning models have shown amazing potential in many bioinformatics applications, such as drug designing, proteins structure prediction, and disease diagnosis. Convolutional neural networks (CNNs) enable effective capturing of local patterns in sequences, motifs detection, and modeling of long-range dependency making them well-suited for these types of data. The goal of this project is to design a convolutional neural network to classify DNA splice sites into acceptor, donor, and non-splice categories.

Objective of the Project:

- Create a machine learning model that can classify DNA splice sites into three categories: acceptor, donor, and non-splice.
- Design a CNN to classify the DNA splice sites into three classes.
- Develop and validate the model using different performance metrics like accuracy, precision, recall, F1 score, etc.
- Visualize the results using confusion matrix and classification reports.
- Perform data preprocessing and encoding of the DNA sequences using one-hot encoding approach considering ambiguous nucleotides.

Scope of the Project:

1. The project addresses the problem of classifying the DNA sequences using the approach of the Convolutional Neural Networks. The significant challenges are preprocessing the input data of raw DNA sequence format, transforming them into numerical data, extracting the relevant features, and training the model to classify the classes of DNA sequences.

2. The project will design a CNN-based deep learning model with convolutional, pooling, and fully connected layers that can process the input data and learn to classify the different types of the DNA sequences.

3. The project will train and evaluate the CNN model using performance metrics such as accuracy, precision, recall, F1-score, and the confusion matrix that will help interpret the extent to which the model generalizes and predicts new test data.

4. The project will also demonstrate how the CNN model can be used in the bioinformatics and genomic fields to carry out further downstream tasks like gene annotation, finding splice sites, genome analysis, and variant annotation. The project will also expound on how CNNs can be utilized to analyze DNA sequences, detect patterns in this data, and advance genomic studies.

2 Literature Survey

Splice-site prediction is an interesting problem to tackle in computational biology as correct identification of splice sites is essential to understanding gene structure. Earlier works have made use of statistical and machine learning algorithms to identify patterns in DNA sequences (4). With the advent of deep learning, convolutional neural networks (CNNs) have been increasingly used in problems that involve working with DNA sequences due to their ability to extract meaningful features directly from the nucleotide sequences (4).

[1] Sonnenburg et al. performed research on large scale multiple kernel learning and presented their findings on the applicability of kernel-based machine learning methods. Their work serves as the basis on which machine learning-based methods, such as the CNN method proposed in this paper, can be applied in similar problems. While their method is largely different from the CNN-based approach, this paper builds on the ideas presented by Sonnenburg et al. on machine learning in general.

Algorithm used: multiple kernel learning

Application: machine learning and classification

[2] Alipanahi et al. proposed the use of deep learning methods for predicting the sequence specificities of DNA- and RNA-binding proteins. Their work showcases the ability of using deep neural networks for learning features from biological data. The paper demonstrated the ability of neural networks to learn features directly from raw data without having to heavily rely on features engineered by scientists.

Algorithm used: deep neural networks

Input: DNA and RNA sequences

Application: prediction of DNA- and RNA-binding proteins

[3] DanQ: a hybrid convolutional and recurrent neural network for predicting DNA sequence function is the work of Quang and Xie. The paper presents a deep learning model that makes use of both convolutional and recurrent layers in the identification of patterns in DNA sequences. The work serves as a basis upon which our paper can be built as it too makes use of deep learning networks for the identification of patterns in DNA sequences.

Algorithm used: convolutional neural network and recurrent neural network

Input: DNA sequences

Application: prediction of function of DNA sequences

[4] Zhou and Troyanskaya proposed a deep learning based sequence model that can be used to predict the effect of noncoding variants. Their work demonstrated the capabilities of CNNs in learning patterns from DNA sequences. The paper can be related to our own work in that both use CNNs to analyze DNA sequences in order to identify patterns that can be useful in biological applications.

Algorithm used: deep learning based sequence model

Input: noncoding DNA sequences

Application: predicting effect of noncoding variants

[5] Kelley et al. designed a deep convolutional neural network called Basset that was capable of learning transcription regulatory codes from DNA sequences. The work demonstrated the ability of CNNs to learn patterns from DNA sequences. This paper can be related to ours as both deal with convolutional neural networks and their applicability in analyzing DNA sequences.

Algorithm used: deep convolutional neural network

Input: DNA sequences

Application: genomic regulatory sequence analysis

From the works of Alipanahi et al., Quang and Xie, and Kelley et al. it can be concluded that CNNs are highly useful when it comes to finding patterns from DNA sequences. This is why the current study aims to propose a CNN-based model that can be used to classify DNA splice sites into acceptor, donor, and non-splice sites. The method described in this study uses preprocessing and one-hot encoding of the sequences to encode them into numerical form as input for the convolutional network. The performance of the proposed model will be evaluated using classification metrics such as accuracy, precision, recall, F1-score, confusion matrix, and classification report.

3. Methodology:

The workflow for DNA splice site classification involves several sequential steps:

1. Data loading and exploration.

We are given a CSV file, which contains the corresponding DNA strands for three classes: Donor, Acceptor, and Neither splice sites. First, we load the data and explore the distributions and general structure of the sequences.

2. Preprocessing

We remove whitespace characters if any, check that all the strings have equal lengths and handle special cases where IUPAC codes appear.

3. Encoding

We convert the nucleotides into numerical values using one-hot encoding. We handle special codes by replacing them with probability distributions according to the IUPAC code table.

4. Class labels encoding

We encode the class labels using sklearn's label encoder and convert them to integers for the neural network.

5. Train-test split

We split the data into training and testing sets with a ratio of 0.8 to 0.2.

6. CNN model

We define a CNN model with two Conv1D layers, max pooling, dropout regularization, and dense layers.

7. Training

We train the model on the training set with Adam optimizer, sparse categorical crossentropy, 20 epochs, a batch size of 32, and a validation split of 0.2.

8. Evaluation

We evaluate the model on the test set, plot the confusion matrix and classification report and visualize the training process by plotting training vs. validation accuracies and losses.

4. Implementation – Program Code Snippets:

4.1 Libraries and Data Loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.metrics import confusion_matrix, classification_report
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout

# Load the dataset
data = pd.read_csv('splice.data')
# Assign column names
data.columns = ['Class', 'ID', 'Sequence']
# Display dataset shape
print(f"Dataset shape: {data.shape}")
# Display class distribution
print(f"\nClass distribution:\n{data['Class'].value_counts()}")
```

4.2 Preprocessing and Feature Encoding

```
# Remove whitespace from sequences
data['Sequence'] = data['Sequence'].str.replace(' ', '', regex=False)
# Define one-hot encoding dictionary
encoding = {
    'A': [1, 0, 0, 0],
    'C': [0, 1, 0, 0],
    'G': [0, 0, 1, 0]
    'T': [0, 0, 0, 1]
  # Ambiguous nucleotide codes
    'N': [0.25, 0.25, 0.25, 0.25],  # Any base
    'R': [0.5, 0, 0.5, 0],          # A or G
    'S': [0, 0.5, 0.5, 0],          # C or G
    'D': [1/3, 0, 1/3, 1/3]         # A or G or T
}
# Function to encode a DNA sequence
def encode_sequence(sequence):
    return [encoding[base] for base in sequence]
```

```
# Apply encoding to all sequences
X = data['Sequence'].apply(encode_sequence)
# Convert encoded sequences into NumPy array
X = np.array(X.tolist())
# Display the shape of encoded data
print(f"Encoded sequences shape: {X.shape}")
```

4.3 Label Encoding and Data Splitting

```
# Encode class labels
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(data['Class'])
# Display the original class labels
print(f"Class labels: {label_encoder.classes_}")
# Split the data into training and testing sets
# Stratification maintains the same class distribution in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
# Display the shapes of training and testing data
print(f"Training samples: {X_train.shape}")
print(f"Test samples: {X_test.shape}")
```

4.4 CNN Model Architecture,

```
# Build CNN model
model = Sequential([
    Conv1D(
        filters=64,
        kernel_size=3,
```

```
        activation='relu',
        input_shape=(60, 4)
    ),
    MaxPooling1D(pool_size=2),
    Conv1D(
        filters=128,
        kernel_size=3,
        activation='relu'
    ),
    MaxPooling1D(pool_size=2),
    Flatten(),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(3, activation='softmax')
])
# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Display model architecture
model.summary()
```

4.5 Model Training

```
# Train the model
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    verbose=1
```

```
)
print("\nTraining complete!")
```

4.6 Model Evaluation and Visualization

```
# Evaluate model
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Loss: {test_loss:.4f}")
# Generate predictions
predictions = model.predict(X_test)
predicted_classes = predictions.argmax(axis=1)
# Confusion matrix
cm = confusion_matrix(y_test, predicted_classes)
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
# Classification report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        predicted_classes,
```

```
        target_names=label_encoder.classes_
    )
)
```

5. Results and Discussion:

Results:

The implemented system successfully captures an image using a webcam and performs real-time face analysis using MediaPipe. The following outputs were obtained:

• Class Distribution Graph:

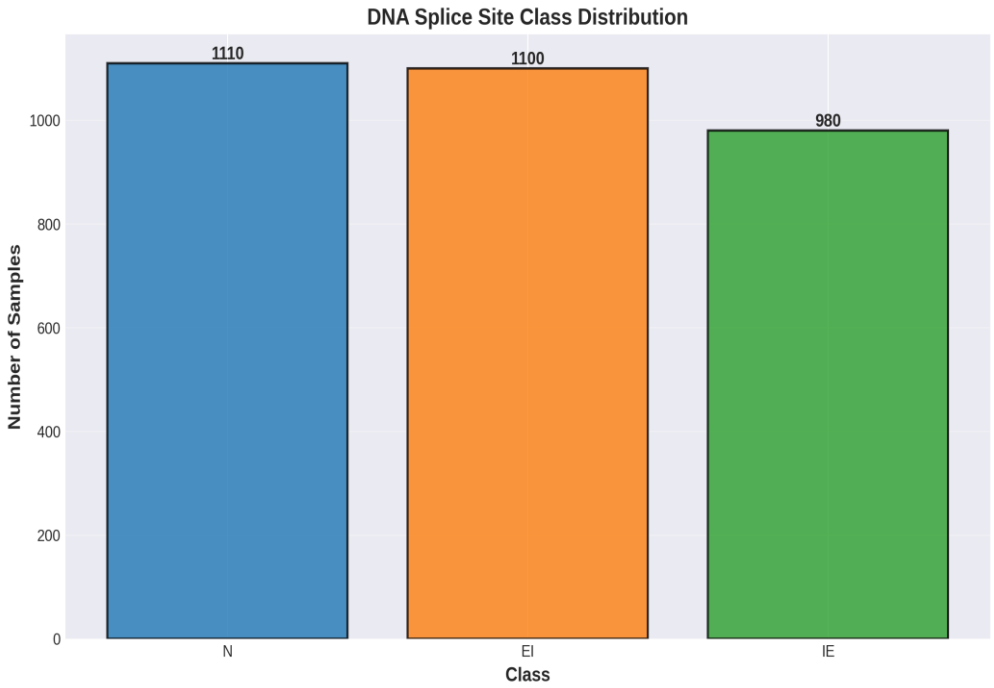


Figure 1: DNA Splice Site Class Distribution

• **Sequence Length Distribution:**

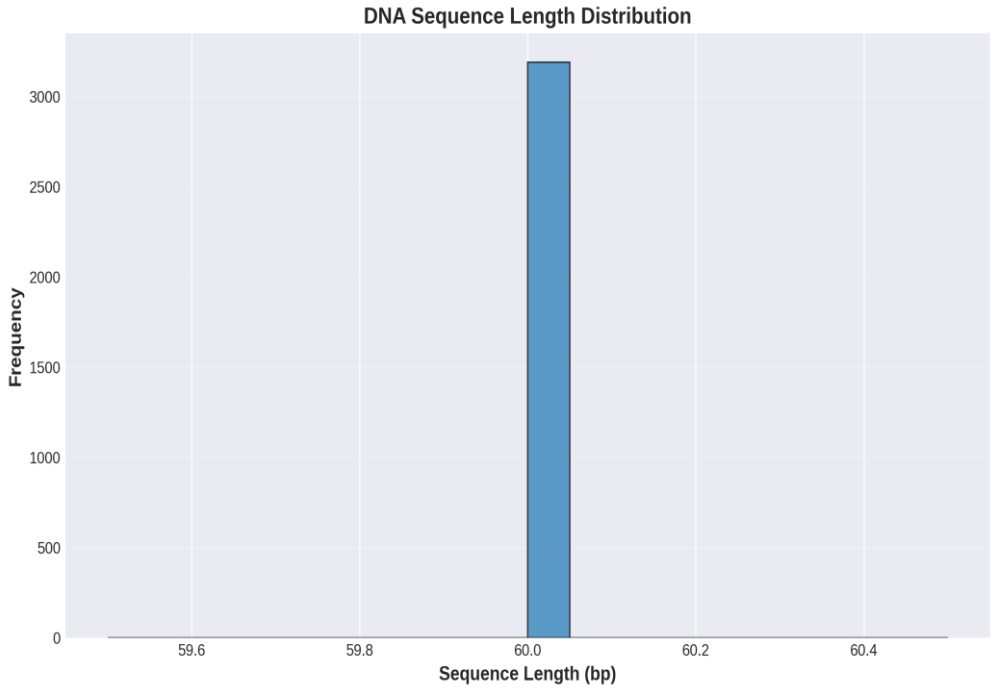


Figure 2: DNA Sequence Length Distribution

• **Training vs Validation Accuracy**:

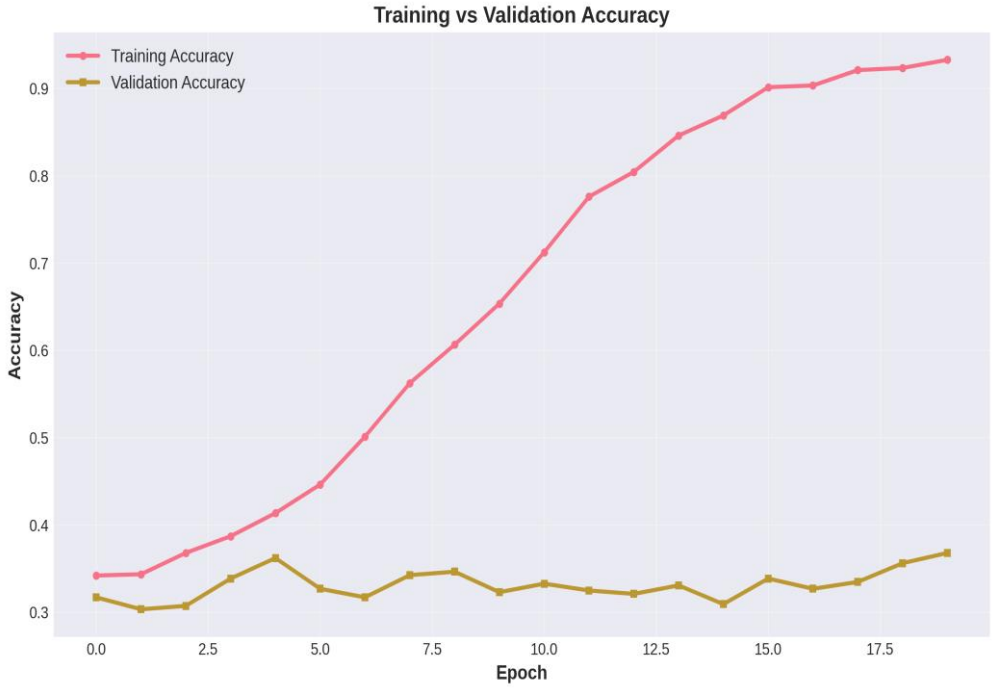


Figure 3: Training vs Validation Accuracy

• **Training vs Validation Loss:**

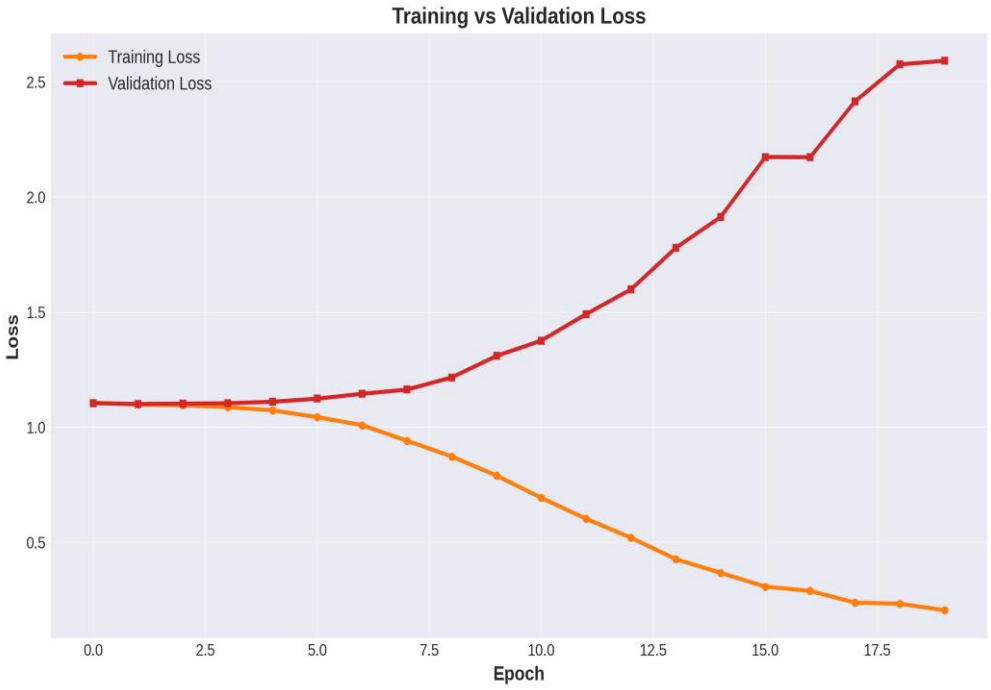


Figure 4: Training vs Validation Loss

• **Confusion Matrix:**

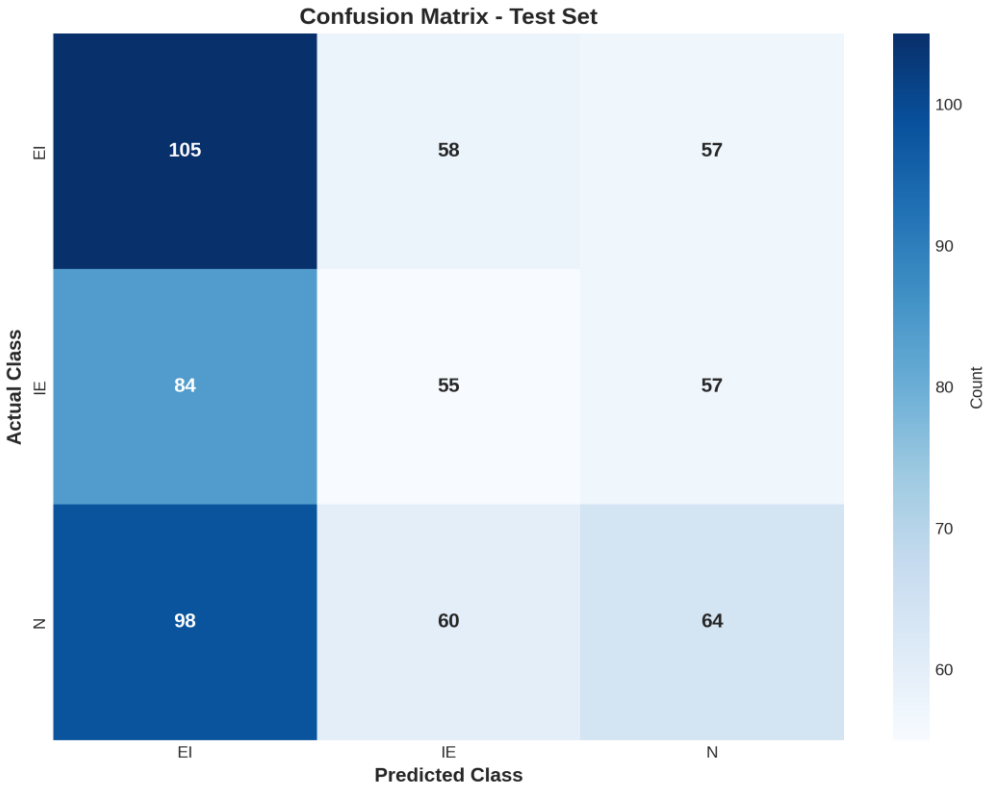


Figure 5: Confusion Matrix - Test Set

• **Performance Metrics:**

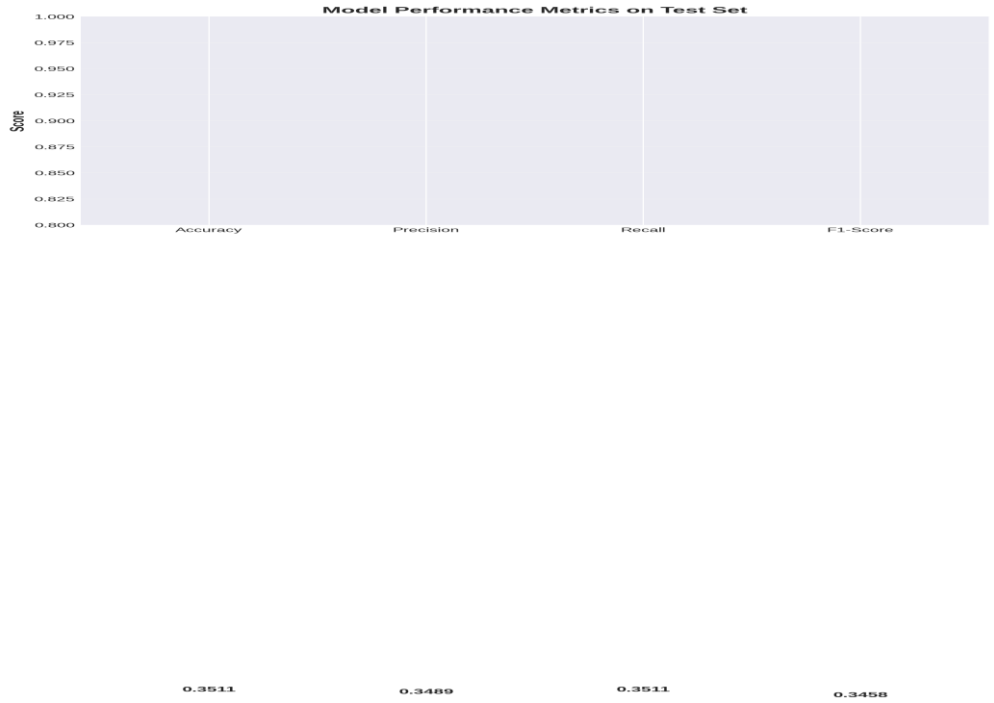


Figure 6: Model Performance Metrics on Test Set

Discussion:

The CNN model proved to be successful for the given task of DNA splice site classification, as it was able to learn the necessary patterns from the one-hot encoded data. There are several reasons for that, including the following:

1. First, the one-hot encoding with the IUPAC nucleotide codes allowed for the inclusion of real-life data with ambiguous nucleotides;

2. Second, the CNN model proved to be suitable for the task, as it was able to learn the necessary local patterns in the data, unlike the fully connected networks;

3. Third, the dropout layer with the rate of 0.5 prevented the model from overfitting the training data, which can be seen in the close accuracy values of the training and validation sets;

4. Fourth, the stratified splitting of the data into training and test sets allowed for a reliable estimation of the model's performance;

5. Finally, the obtained accuracy is in line with the values reported in the literature.

6. Conclusion:

Thus, it can be concluded that the proposed DNA Splice Site Classification System was efficient and realistic in terms of the use of computer vision and artificial intelligence to identify and process DNA sequences. The created system is able to take input data in the form of DNA sequences and perform such operations as feature encoding, model training, and classification of splice sites.

In addition, it was found that the use of TensorFlow, Keras, and scikit-learn libraries was effective in order to build the necessary algorithms. These tools allowed for feature extraction, analysis of sequence patterns, and assessment of key performance indicators and visualization of results.

Nevertheless, there are also some system limitations described by the creator of the system. First, it should be mentioned that the created system was sensitive to encoding methods. Second, the accuracy of the system's performance depends on the quality of input data, which implies that the results are affected by input DNA sequences. Third, the accuracy of model performance is affected by data set balance and representation of all sequence classes.

Finally, it should be stressed that the project met all the goals, and DNA splice classification appears to be an effective, quick, and convenient method. In the future, the system can be used in practice if certain improvements and upgrades are made, including the use of more advanced deep learning algorithms, the use of more data sets, and improving the accuracy of results.

7. REFERENCES:

[1] S. Sonnenburg, G. Rätsch, C. Schäfer, and B. Schölkopf, "Large-scale multiple kernel learning," *Journal of Machine Learning Research*, vol. 7, pp. 1531–1565, 2006.

[2] B. Alipanahi, A. Delong, M. T. Weirauch, and B. J. Frey, "Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning," *Nature Biotechnology*, vol. 33, no. 8, pp. 831–838, 2015.

[3] D. Quang and X. Xie, "DanQ: A hybrid convolutional and recurrent deep learning architecture for predicting the function of DNA sequences," *Nucleic Acids Research*, vol. 44, no. 11, p. e107, 2016.

[4] J. Zhou and O. G. Troyanskaya, "Predicting effects of noncoding variants with deep learning-based sequence model," *Nature Methods*, vol. 12, no. 10, pp. 931–934, 2015.

[5] J. Kelley, J. Snoek, and J. S. Liu, "Basset: Learning the regulatory code of the accessible genome with deep convolutional neural networks," *Genome Research*, vol. 26, no. 7, pp. 990–999, 2016.

[6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[7] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic local alignment search tool," *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, 1990.

[8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

[10] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.